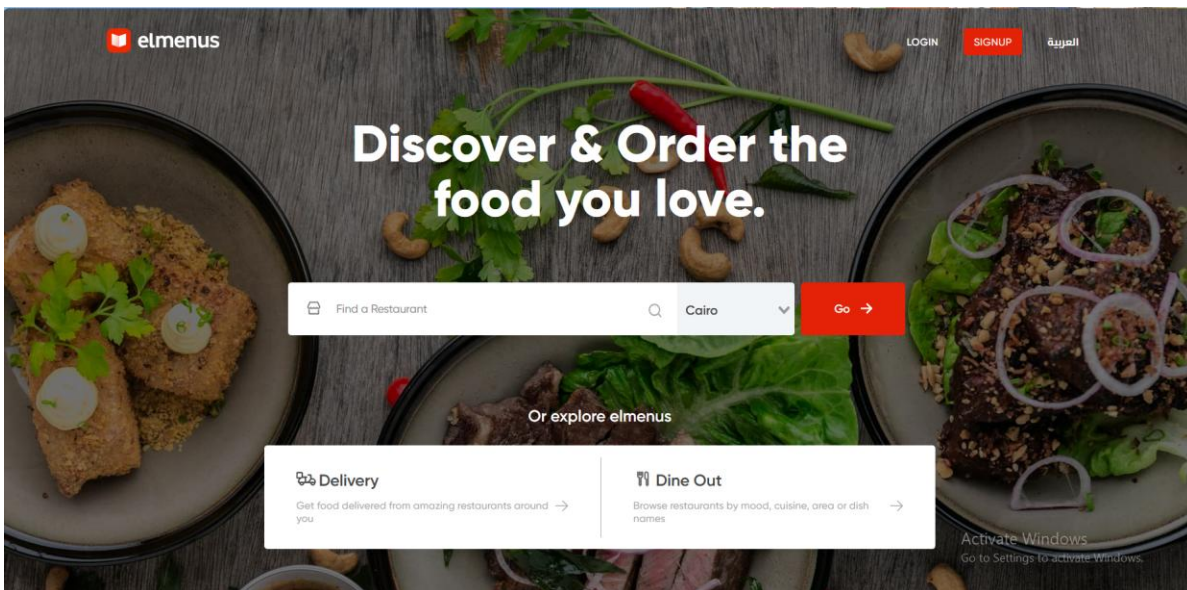
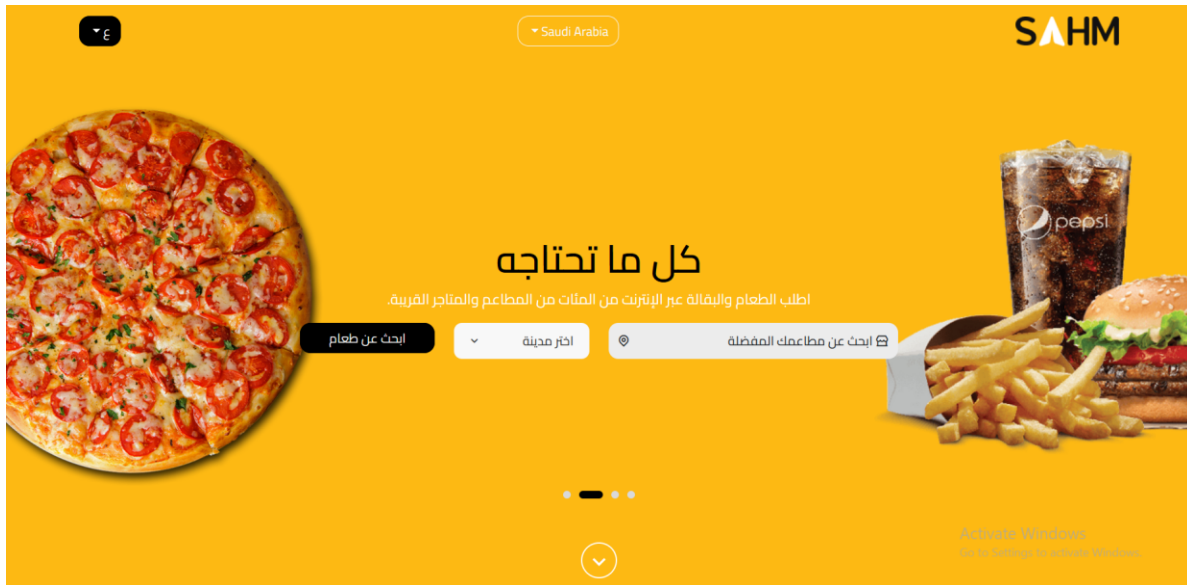


# Test Plan for Elmenus

**System Under Test:** Elmenus (elmenus.com web + Elmenus mobile app, Android)



**Tester role:** External QA engineer, black-box testing the public product

**Assumptions:** All API schemas, internal behaviors, and reproduction steps reflect what an external tester can observe or reasonably infer.

The Elmenus API is secured and not publicly documented; endpoints, request structures, and expected responses in this test plan are based on network traffic observed via Firefox DevTools and represent the best available external approximation. As a result, a full end-to-end API test execution was not possible.

## 1. Objectives

---

- ➔ Verify the revenue flow (browse → cart → checkout → pay → track → deliver) works end-to-end
- ➔ Surface the high-impact edge cases food delivery is uniquely exposed to (failed payments, network drops, items out of stock, restaurants offline, delivery-zone edges, ETA delays)

## 2. Test Plan

---

### 2.1. Scope

- ➔ *Mobile app (Android): Browse, search, cart, checkout, payment flows (card + cash on delivery + wallet), order tracking, push notifications, app permissions (location, notifications), offline behavior.*
- ➔ *Web (elmenus.com responsive): Restaurant discovery, menu browsing, cart and checkout on desktop and mobile web breakpoints, guest vs logged-in flows.*
- ➔ *APIs (backend): Restaurant discovery, Restaurant listing/search, menu fetch.*
- ➔ *Out of scope: Restaurant-side POS / dispatcher tooling (not publicly accessible), payment gateway internals (Fawry / card processor), SMS/email delivery infrastructure.*

### 2.2. Testing types

- ➔ **Functional** (API & UI): Core user journeys (place order, apply promo, change address, cancel order).
  - ➔ **Regression**: Smoke suites covering the most critical paths, run before every release. Full regression on major releases & Focused regression on any module touched by a fix.
  - ➔ **Integration**: menu sync (restaurant catalog → customer app), order state sync, payment gateway callback handling, location service → delivery zone validation.
- 

## 3. Entry & Exit criteria

---

### 3.1. Entry criteria

- ➔ Build deployed to staging/test **environment** with version tag.
- ➔ **Smoke test** on the build passes (app launches, login works, one test order completes end-to-end).
- ➔ **Test data ready**: test accounts, test restaurants, test promo codes, test payment cards.
- ➔ **APIs** reachable from test environment with valid test credentials

### 3.2. Exit criteria

- ➔ 100% of P0 test cases executed, 100% passed
  - ➔ ≥95% of P1 test cases executed, ≥98% passed
  - ➔ No open P0 or P1 defects
  - ➔ All open P2 defects have an owner and a triage decision (fix / defer / won't fix).
  - ➔ Regression suite green on the release candidate
  - ➔ Sign-off from QA lead + product owner.
- 

### 4. Risk areas:

| Risk                                           | Why it matters                                                       |
|------------------------------------------------|----------------------------------------------------------------------|
| Payment failures                               | Direct revenue impact; double charge is a customer trust event.      |
| End-to-end order flow                          | Any break means no revenue; visible to every user.                   |
| Restaurant menu sync                           | Wrong price or stale availability.                                   |
| Delivery tracking / ETA                        | Inaccurate ETA is #1 UX complaint in food delivery                   |
| Address & geolocation                          | Wrong delivery zone.                                                 |
| Promo codes                                    | Pricing engine bugs leak money                                       |
| Sync between app and restaurant-facing systems | Order accepted in app but never reaches restaurant = silent failure. |
| Multi-language / RTL (Arabic)                  | Egyptian market so, Arabic-first.                                    |

### 5. Prioritization strategy

---

- ➔ **P0 (Core Business Functions):** Blocks order completion or causes financial loss for any user.  
**Example:** checkout button doesn't respond; cart total miscalculates promo; payment charged but order not created.
  - ➔ **P1 (Major degradation):** Significantly degrades a core flow but workaround exists.  
**Example:** tracking screen doesn't refresh; user can still place orders; address autocomplete broken in one city.
  - ➔ **P2 (Minor / Cosmetic):** Minor UX issue or affects an edge segment.  
**Example:** Text misalignment; obscure deep link breaks; non-default language has a typo.
  - ➔ For test execution, all **P0** cases run on **every build (Test env.)**, **P1** on every **release (Production env.)**, **P2** in **scheduled full regression** cycles only.
-

## 6. Test Case Execution

- ➔ Test cases are written and maintained in structured sheets with all required attributes (e.g., ID, priority, preconditions, steps, expected results, and status).
- ➔ Test cases are executed manually, except for API test cases, which are executed using Postman.
- ➔ After execution, the test case status is updated (e.g., Pass, Fail, Blocked).
- ➔ Any defects are reported and traced back to their corresponding failed test cases.

## 7. Test Environment

| Layer     | Setup                                                     |
|-----------|-----------------------------------------------------------|
| Web       | Mozilla Firefox (latest) on Windows 11                    |
| Mobile    | Android device (Android 11+), Elmenus app from Play Store |
| API       | Postman (latest) on Windows 11                            |
| Network   | Wi-Fi + mobile 4G/5G                                      |
| Languages | English + Arabic (for RTL testing)                        |
| Accounts  | One personal Elmenus account on production                |
| Payment   | Personal debit card + cash on delivery                    |

### 7.1. Test Environment Assumptions

- ➔ Testing runs against production — no staging or QA environment available.
- ➔ Play Store app version = current public release.
- ➔ APIs follow REST conventions with JSON and Bearer-token auth.
- ➔ Test accounts have no special privileges — same access a real user has.
- ➔ Mansoura location is in an active delivery zone; mock-location used for out-of-zone tests.

### 7.2. Test Environment Limitations

| Limitation                                            | Impact                                                                       |
|-------------------------------------------------------|------------------------------------------------------------------------------|
| Android only — no iOS device                          | iOS-specific bugs (Safari, push behavior) not covered                        |
| Firefox only — no Chrome/Edge/Safari                  | Cross-browser bugs may be missed                                             |
| Production only — no sandbox                          | No destructive testing; real money at risk; sandbox card numbers unusable    |
| No backend access — logs, DB, admin tools unavailable | Diagnosis limited to user-visible behavior + network inspector               |
| No restaurant-ops access                              | Cannot trigger menu changes or status transitions on demand (affects TC-003) |
| No payment sandbox                                    | Decline tests (TC-004) require a real low-balance card or get Blocked        |
| Manual execution only                                 | No automation — each test runs once unless reproducing a failure             |
| Single location (Mansoura)                            | Cross-city geofencing edge cases need mock-location                          |